Homework 1 - Multiple Processes (5 points)

Due: March 28, 11:30 PM

Submission Requirements

You need to submit a compressed archive of your homework directory. Under your homework directory, you should include:

- All your source code.
- A Makefile for compiling the program.
- A README file that describes how to compile and test your program.

Before submitting, create a gzipped tarball (.tar.gz) of your entire homework directory using the following command:

tar -czvf username-homework-1.tar.gz your_homework_directory/

where username is your NETID.

Submit the tarball on Canvas.

Problem

Level 1

Implement a program that calculates the sum of numbers from 1 to 100 using 2 to 5 processes.

For example, if using 2 processes:

- The first process can compute: 1+2+3+...+50
- The second process can compute: 51+52+53+...+100

Requirements:

- Since partial sums will exceed 255, you must use an appropriate Inter-Process Communication (IPC) mechanism to pass results from child processes to the parent.
- The parent process should wait for all child processes to complete, retrieve the results using IPC, and print the final sum.
- Ensure your code is readable and robust.
- Print standard Linux error messages if an error occurs (e.g., use perror("Process creation failed") if a child process cannot be created).
- Once the calculation is complete, the main process should print the final summation result.

Level 2

Enhance your program to allow user input for:

- The end number n
- The number of processes

For example, the command:

```
./main 200 10
```

should calculate the sum 1 + 2 + ... + 200 using 10 processes for subset additions.

Input Constraints:

- The end number **n** must be between 10 and 1000 (inclusive).
- The number of processes must be between 2 and n/2 (inclusive), with a maximum of 100 to avoid system overload.
- If the input values are out of range, the program should exit with an error message.

Additional Considerations

- The assignment focuses on multiple processes. Do not use the direct mathematical formula n(n-1)/2 to compute the sum.
- You must implement an IPC mechanism to allow child processes to return their computed partial sums to the parent, since exit() can only return value 0-255.
- Acceptable IPC methods include:
  - Pipes (pipe() or mkfifo) – Direct communication between parent and child processes.
  - Shared memory – Each process writes its result to a shared memory segment.
  - Message queues – Child processes send messages to the parent containing their sums.
- The parent process must synchronize with the termination of all child processes before printing the final result.